

# Class 7: Functions and Good Practices

Data-driven Systems Engineering

# Agenda

- Writing small, single-purpose functions.
- Multiple return values, and functions as values themselves.
- Splitting code across files: `modules`, `import` versus `from...import`.
- The Zen of Python and what “good practice” means in this course.
- Naming, readability, refactoring, and a first taste of testing.

## Learning goals

- Write functions with clear inputs, outputs, and a single responsibility.
- Return and unpack multiple values from one function.
- Organize functions into a module and import them elsewhere.
- Apply naming and structure conventions that make code easier to read and change.
- Recognize refactoring opportunities and describe a function's behavior precisely enough to test it.

# Course Map and Running Cases



## Today in the course

**Focus:** Functions as the unit of reuse, and the habits that keep a growing codebase readable.

**Bridge:** Class 4 introduced `def`; this class asks what makes a function — and the file around it — genuinely good.

## Fraud case

The fraud model's training script and API are each only a handful of functions; their clarity is what makes the pipeline trustworthy.

## Pasteurization case

The pasteurization simulator, 'pasteurization/synth\_sensors.py', is built almost entirely from small, typed, single-purpose functions — our main case study today.

**Course thread:** Good functions are what let a one-person script become a codebase other people, including future you, can safely change.

## Small, single-purpose functions

```
def ramp(start: float, end: float, n: int) -> np.ndarray:  
    return np.linspace(start, end, n)  
  
def smooth_noise(n: int, scale: float) -> np.ndarray:  
    steps = rng.normal(0, scale, size=n)  
    return np.cumsum(steps) / max(n, 1)  
  
def mu_of_T(T: np.ndarray) -> np.ndarray:  
    return np.interp(T, [10, 72], [VISC_COLD, VISC_HOT])
```

- From 'pasteurization/synth\_sensors.py': each function does exactly one thing and says so in its name and type hints.
- Type hints (start: float, -> np.ndarray) are optional in Python but document intent for free.

# Multiple return values

```
def simulate_batch(bid, start_time=0.0):  
    ...  
    return df, t  
  
df, t = simulate_batch(BATCH_ID)
```

- Python functions can return several values at once, packed as a tuple and unpacked on the caller's side.
- 'simulate\_batch' in 'synth\_sensors.py' returns both the simulated batch (df) and the elapsed time (t) — both needed by the caller, neither deserving its own function.

# Functions as values

```
SEGMENT_FUNCS = {  
    "Idle": seg_idle,  
    "Fill": seg_fill,  
    "HeatUp": seg_heatup,  
    "Hold": seg_hold,  
    "Cool": seg_cool,  
    "Discharge": seg_discharge,  
}
```

- From 'pasteurization/serving.py': a dictionary whose values are functions, not numbers or strings.
- This replaces a long `if/elif` chain with a single lookup — functions are ordinary values in Python, storable and passable like anything else.

# Splitting code into modules

```
# pasteurization/serving.py
from synth_sensors import (
    seg_idle, seg_fill, seg_heatup, seg_hold, seg_cool, seg_discharge,
    PROD_RANGES, DT, rng, mu_of_T
)
```

- 'synth\_sensors.py' is a module: a file of related functions and constants that other files import from.
- `import x` loads the whole module under one name; `from x import a, b` pulls specific names directly into scope.
- As a script grows, moving related functions into their own file is usually the first real refactor a project needs.

# Good practice: what it is, and why it evolved

- Best practices come from decades of programmer experience, not arbitrary style preference.
- They increase productivity and decrease the mental overhead of reading someone else's code — including your own, months later.
- Python encodes much of its own philosophy in the Zen of Python (`import this`).

# The Zen of Python, selected

- “Explicit is better than implicit.”
- “Readability counts.”
- “Simple is better than complex.”
- “Errors should never pass silently. Unless explicitly silenced.”
- “In the face of ambiguity, refuse the temptation to guess.”

## Course translation

Every one of these lines is really an argument for naming things clearly, handling failure on purpose, and not being clever for its own sake.

# Naming conventions

| Style                   | Used for                                                                              |
|-------------------------|---------------------------------------------------------------------------------------|
| <code>snake_case</code> | Ordinary variables and function names, e.g. <code>smooth_noise</code> .               |
| <code>UPPER_CASE</code> | Module-level constants, e.g. <code>N_BATCHES</code> , <code>MODEL_ACCURACY</code> .   |
| <code>PascalCase</code> | Class names, e.g. <code>RandomForestClassifier</code> , <code>StandardScaler</code> . |
| <code>Leading _</code>  | A hint that a name is private or internal to a module.                                |

- Every one of these examples already exists somewhere in this repository — the convention is not just theory.

## Deeply nested

- if data is valid:
  - if model is loaded:
    - run prediction

## Guard clauses

- if not valid: return
- if not loaded: return
- run prediction

### Idea

Handle the exceptional case first and return early, so the main logic is not buried three levels deep.

- Use `enumerate()` instead of a manual counter when you need both index and value.
- Comments should explain *why*, not restate *what* the code already says.
- Keep comments up to date — an outdated comment is worse than no comment.
- Prefer a well-named variable over a comment explaining a badly-named one.

# A first taste of testing

```
def ramp(start: float, end: float, n: int) -> np.ndarray:  
    return np.linspace(start, end, n)  
  
assert list(ramp(0.0, 10.0, 2)) == [0.0, 10.0]
```

- Black-box testing means checking a function's output for known inputs, without caring how it is implemented inside.
- A single-purpose, typed function like `ramp` is exactly the kind of function that is easy to test this way.
- Class 11 formalizes this idea further once scikit-learn's own API enters the picture.

# How the repo already uses today's building blocks

## Repo activity

- 'pasteurization/synth\_sensors.py' is a full module of small, typed, single-purpose functions — read it end to end as a worked example of today's lecture.
- 'pasteurization/serving.py' imports selectively from that module and stores functions inside a dictionary.
- Pick one function from 'synth\_sensors.py' and rewrite its one-line docstring in your own words.

### Question

'SEGMENT\_FUNCS' replaces an `if/elif` chain with a dictionary lookup. What would you lose, if anything, by keeping the `if/elif` chain instead?

- Encourages weighing readability and extensibility against a construct students may find less familiar than the alternative.

## Papers and materials

Sommerville, *Software Engineering*; Chip Huyen, *Designing Machine Learning Systems*; [scikit-learn documentation](#).

# From This Class to the Next

## What stays with us

- A good function is small, named clearly, and returns exactly what its caller needs — one value or several.
- Modules let related functions live together and be imported where needed.
- Naming, guard clauses, and small testable functions are habits, not one-time decisions.

## Project use

Before your project script passes 50 lines, check whether it should already be split into functions, and those functions into a module.

## Next connection

Class 8 goes one level further: bundling functions and the data they operate on together, inside a class.

**Course thread:** Functions group behavior; next class groups behavior with the data it belongs to.